Chapter 11 – Greedy Algorithms

By Dr. S. Sridhar

Greedy Algorithms

- A greedy approach is a problem-solving strategy like the divide and conquer approach.
  - It is applied to optimization problems.
- What is an optimization problem?
  - An optimization problem is associated with an objective function called a value, a predicate P representing feasibility criteria, a solution space of all possible solutions U, and an extremum (maximum or minimum) requirement.
- Most greedy problems have ‘n’ inputs
- The problems are often associated with constraints
- Constraints are conditions that affect the choice of decisions
- All potential solutions are called candidate solutions
- Any candidate solution that satisfies the constraints is called a feasible solution
- An optimal function is a feasible solution that maximizes or minimizes the objective function
- Greedy algorithms solve optimization problems exercising locally optimal decisions at every step that can lead to a globally optimum solution.

Greedy Approach vs. Divide and Conquer Approach

Greedy approach	Divide and conquer approach
An optimization technique aims to find an optimal solution	The divide and conquer approach is for finding a general solution
Constructs optimal solutions by combining feasible solutions	It uses a divide-and-conquer approach to split the problem into subproblems and solve them. The final solution is then by combining solutions of the subproblems.
The greedy approach always avoids recomputation	It uses recomputation very much
Iterative in nature	Recursive in nature
Slower	Faster

Greedy General Algorithm

Algorithm greedy(A, n)

```
%% Input: Array A[1 .. n]
%% Output: Solution of the problem
      SolutionSet = Null                                    %% initially, the solution is null
      i = 1
      while (i ≤ n) do
           x = Select a best candidate solution x from A    %% Selection procedure
           if (feasible(SolutionSet,x))                     %% Feasible if constraints are satisfied
                SolutionSet = SolutionSet + x
           end if
                i = i + 1
      End while
      %% Check if the solution to the given problem is obtained
           Return(SolutionSet)
     end while
End
```

Components of Greedy Algorithm

- Problem Identification: An optimization problem often needs a greedy solution
- Objective Function with maximization or minimization specification
- Constraints of the problem
- Decision Space: Set of all possible decisions. All possible candidate solutions may not be optimal solutions
- Selection criteria: Used to select the best locally optimal decision
- Iterative decision making
- Solutions are verified by checking and returned as the solutions to the given problem

Components of Greedy Approach

Box 11.2 **Components of greedy approach**

The important components of the greedy approach are given below:

Objective function The greedy problem objective should be stated as an objective function. The objective function should be maximized or minimized based on the given problem.

Generating multiple candidate solutions A greedy problem may have *N* inputs or candidate solutions (i.e., possible solutions for the given problem).

Selection procedure A selection procedure must exist for a greedy algorithm to choose the next item to be considered. The selection must be done based on some greedy, locally optimal criteria.

Feasibility check Determines if the selected item is feasible per the constraint. If the constraint is not fulfilled, the selected item is rejected. If the constraints are satisfied, the item is added to the set of feasible solutions.

Solution check This checks whether the partial solutions constitute a global solution for the given problem for a given instance, and if so, a solution is returned.

Properties of Greedy Algorithms

- **Optimum substructure:** A problem is said to have an optimum substructure if it possesses the property that the global solution within it contains optimal subsolutions to the subproblems.
- Greedy Choice Property: A globally optimal solution can be obtained by combining locally optimal decisions
  - Selected decisions or choices must have the following characteristics of
    - Feasibility, Optimality and irrevocability
  - **Feasibility:** Decisions or choices must satisfy the local constraints, optimality
  - **Irrevocability:** The made choice can not be revoked or changed
  - **Optimality:** The chosen decision may not be global optimum but locally optimum

Coin Change Problem

- Objective function: This is to minimize the number of coins for a request.
- Candidate solutions: These involve all possible solutions involving coins of different denominations.
- Selection procedure: Like a greedy man, the selection procedure should look for coins of the largest denomination to fulfil the request and thereby to minimize the number of coins.
- Feasibility check: One would not choose a coin of denomination 50, if the request is less than 50. Hence, the feasibility check should determine the feasibility of solutions. If the choice is not suitable, then the selected item is rejected. Otherwise, it is added to the set of solutions.
- Solution check: Finally, the collected solution is checked and returned as the output.

Algorithm coin_change(A, n, r)

```
%% Input: An array A of denominations A[1 ... n] and requested coin; r is requested coin
%% Output: A set of coins of different denominations that satisfy the given request
Begin
    solution = ϕ     %% Initially the solution set is null as there are no coins added
    sum = 0          %% no coin is dispersed
    while (sum < r) do
        Grab the largest coin x in A       %% Selection procedure
        if ((sum + x) <= r) then           %% feasibility check
              solution = solution ∪ x
              sum = sum + x
        End if
    End while
    return (solution)
End
```

The coin change algorithm is a constant time algorithm $O(1)$

Scheduling Problem

Box 11.4 **Scheduling problem**

Let us visualize the following scenario of a scheduling problem:

In a shop, there is only one clerk. Various customers wait to get the bills for the items they have purchased. This scenario is shown in Fig. 11.4. The customers who have bought one item requires less time for processing and those who have bought a large number of items would require a lot of time for billing. The clerk feels that perhaps a little reordering of the queue might help satisfy all the customers. If so, what would be the best optimal order?

Obviously, if customer 1 purchases 1000 items and customer 2 purchases 5 items, it makes sense to process the bill for customer 2 ahead of customer 1, so as to minimize the collective waiting time of customers.

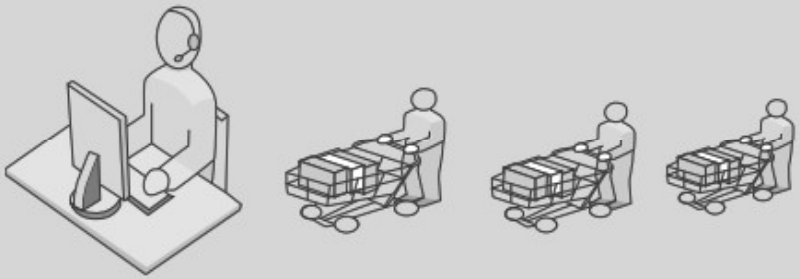


Fig. 11.4 Scheduling problem

Categories of Scheduling Problem

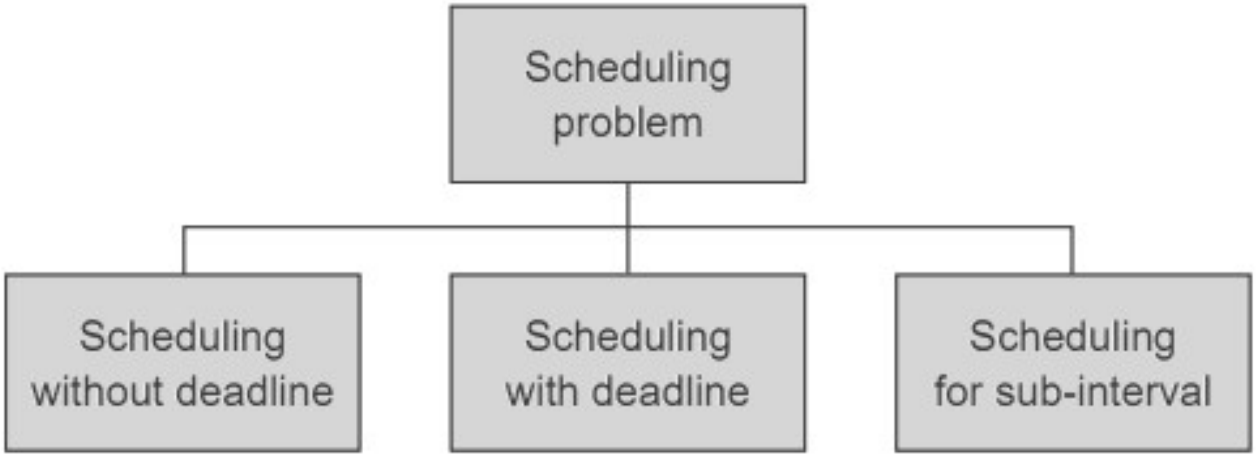


Fig. 11.5 Categories of scheduling problem

Scheduling with Deadline

- Goal: Schedule the tasks such that maximum total profit is obtained without violating the deadline constraints
- This algorithm differs from the previous problem because task completion is associated with profit
- The job must be completed before the deadline to make a profit
  - Otherwise, the job completion does not count or earn any profit at all
- The problem aims to construct a feasible sequence that gives the maximum profit
- A sequence is feasible if all the jobs end by their deadline
- A set of jobs is feasible if at least one sequence is possible
- The sequence associated with the maximum profit is called the optimal sequence
  - The elements that constitute the sequence comprise the optimal set of jobs.

Example: Consider the items and profit as shown below and find the optimal set of jobs that can be scheduled, so that the profit is maximized

Job	Deadline	Profit
1	2	60
2	1	30
3	2	40
4	1	80

Possible job sequence	Total profit obtained
2, 1	90
3, 1 or 1, 3	100
2, 3	70
4, 1	140
4, 3	120

Schedule with deadline - algorithm

Algorithm schedule_with_deadline

```
%% Input: A set S of jobs {1,2,…,n} with deadlines
%% Output: An optimal schedule J
Begin
      S = sorted array of jobs based on profit in non-decreasing order
      i = 2
      J = {1}
      while (i <= n) do                  %% for all jobs do
            select the next job i from the set S
            K = K U job i with job added on the ascending values of deadline      %% Selection procedure
            if (K is feasible), then                                               %% Feasibility check
            J = K
            i = i + 1
      End while
      return(solution)
End
```

Time Complexity: $O(n \log n)$

Activity Selection Problem

- Let us assume a university department has only one auditorium
  - The university has many activities
  - All the activities involve a function that must be conducted in the auditorium
- This kind of problem, where limited resources need to be allotted to more activities optimally, is called an activity-selection problem
- This problem is also known as an interval scheduling problem, as these activities involve time intervals defined by starting and finishing time
- I can be observed that all other problems such as classroom scheduling

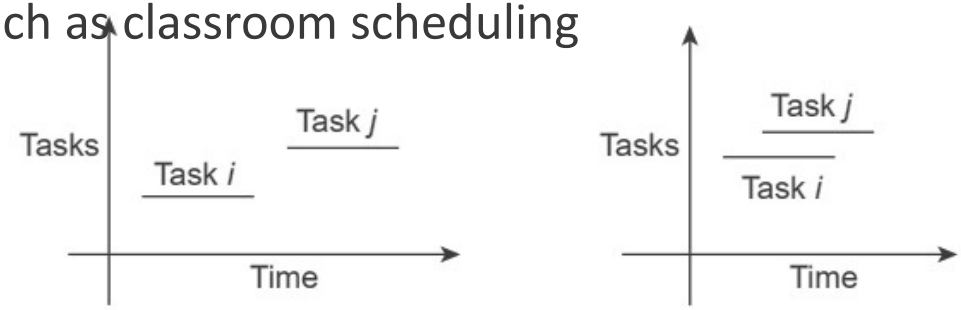


Fig. 11.6 Overlapping and ~~non-~~overlapping of tasks

Greedy-Schedule Algorithm

Algorithm Greedy-schedule(course[1 .. n])

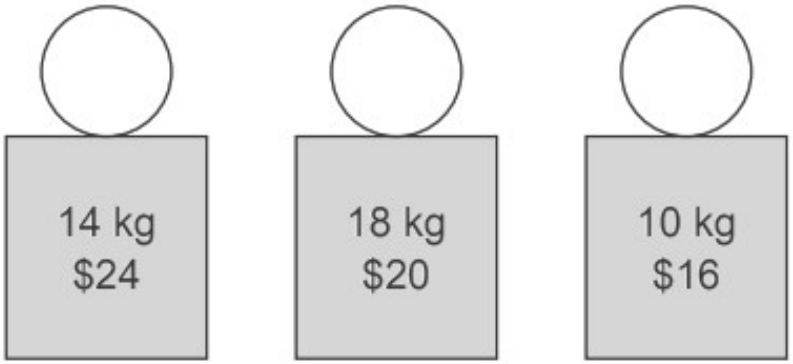
```
%% Input: Array of courses along with start time (s) and finish time (f)
%% Output: Optimal course schedule S
Begin
      Sort all courses based on finishing time in non-decreasing order to A
      A = {set of all sorted courses and renumbered as 1, 2, ..., n}
                                          %% Such that f_1 ≤ f_2 ≤ ··· ≤ f_n
                                          %% Add the first course with the earliest finish time
      S = {1}
      j = 1
      for k = 2 to n do                   %% Check for all the courses
      if s_k ≥ f_j then
                                          %% Check whether the activity is feasible that
                                          %% is the finish time of the current activity
                                          %% less than the start time of the next
      S = S U {a_k}                       %% If feasible, add the current activity
      j = k
      End if
      End for
      return(S)
End
```

Time Complexity: $O(n \log n)$

Knapsack Problem

- The aim of loading problem is to pack objects of different sizes or volumes into a finite number of bins of certain capacity
  - Aim of the problem is often minimization of space or maximization of profit
- No problem if abundant space is available or a few items are to be packed
  - This condition rarely exists
- Loading problems are always considered as challenging problems
- Knapsack is one such loading problem
  - This is a special case of a bin-packing problem
- A bin-packing problem deals with the use of a minimum number of bins
- Bin-packing problems are discussed further in Additional Problems

Example for Knapsack Problem



(a)

Items	Weight	Profit
1	14	24
2	18	20
3	10	16

(b)

Fig. 11.7 Knapsack problem (a) Bottles illustrating items (b) Tabular representation of weights and profits

Solution for the above problem

x_i	x_2	x_3	$\sum x_i w_i$	$\sum p_i x_i$	Comments
1	1/3	0	20	30.66	According to profit
0	5/9	1	20	27.11	By least weight
1	0	3/5	20	33.6	By profit to weight ratio

Complexity:
$O(n)$

Algorithm greedy_knapsack(W, n)

```
%% Input: n items with profit p[1 .. n] → Profit and w[1 .. n] → weight, W is the
%% capacity of the knapsack. It is also assumed that the items are ranked by p_i/w_i
%% Output: Optimal packing order of items stored in solution vector x[1 .. n]
Begin
    // Initialize the solution vector
    for i ← 1 to n do
        x(i) := 0.0;
    End for
    load = 0                        %% Initial weight of knapsack i
    i = 1                           %% start with the first site,
    while ((load <= W) and (i <= n)) do
    if (w_i + load <= W) then
        load = load + w_i           %% Load item fully
        x[i] = 1                    %% Mark in the solution vector that the item is
                                    %% loaded fully
    else
        r = W - load                %% Compute the space left out
        load = load + r/w_i         %% Fill knapsack with the fraction of item
        x[i] = r/w_i                %% Record the amount of item in solution vector
    End if
    End while
    return(x)                       %% Return solution vector
End
```

Optimal Storage Problem

- The optimal storage problem is as follows:
  - Given n programs, the optimal storage tape problem is to store the programs, so that the retrieval time is minimized
  - Let us assume that there are n programs. Each program i has length l_i, where i ranges from 0 to 1
  - All the programs can be stored on a tape if the sum of the lengths of all programs is less than the capacity of the magnetic tape
  - Let the length of the magnetic tape be l
  - The time is directly proportional to the number of programs stored ahead
  - Mean Retrieval Time $MRT = \frac{1}{n}\sum_{1\leq j\leq n} t_j$
  - Objective Function: $\min \sum_{1\leq j\leq n} \sum_{1\leq k\leq j} l_k$

Optimal Storage Algorithm - Greedy Approach

Algorithm optimal_storage(L)

```
%% Input: Array L consists of the length of programs i1,i2,…,in
%% Output: Optimal storage order of programs
Begin
      Sort all the programs based on length
      for i = 1 to n do
        for j = 1 to i
           tj = tj + L(j)
        End for
      End for
          tj = tj/n
      j = 0                        %% j is the next tape
      For i = 1 to n do
           j = (j + 1) mod m       %% m is the number of tapes
      End for
End
```

Time Complexity: $O(n \log n)$

Example : Let n = 3 and the program length be $l_1 = 3$, $l_2 = 4$, and $l_3 = 7$. Find the optimal order for storing the programs using the brute force approach

	Retrieval time	Total	MRT
[1, 2, 3]	3 + [3] + 4 + [3 + 4] + 7	24	8
[1, 3, 2]	3 + [3] + 7 + [3 + 7] + 4	27	9
[2, 1, 3]	4 + [4] + 3 + [4 + 3] + 7	25	8.3
[2, 3, 1]	4 + [4] + 7 + [4 + 7] + 3	29	9.6
[3, 1, 2]	7 + [7] + 3 + [7 + 3] + 4	31	10.3
[3, 2, 1]	7 + [7] + 4 + [7 + 4] + 3	32	10.6

Minimum Spanning Trees

- The MST should have the following properties:
  - MST is connected and has no cycles
  - An MST T has a minimal cost
  - MST T has $n - 1$ edges, where n is the number of vertices of the graph G
  - If K is the set of edges not in the spanning tree, adding an edge from K to T creates a cycle
  - The spanning tree has all the vertices of G and no cycles

Example: Consider graph G shown in Fig. 11.16. Construct an MST for the given graph G using Kruskal's algorithm

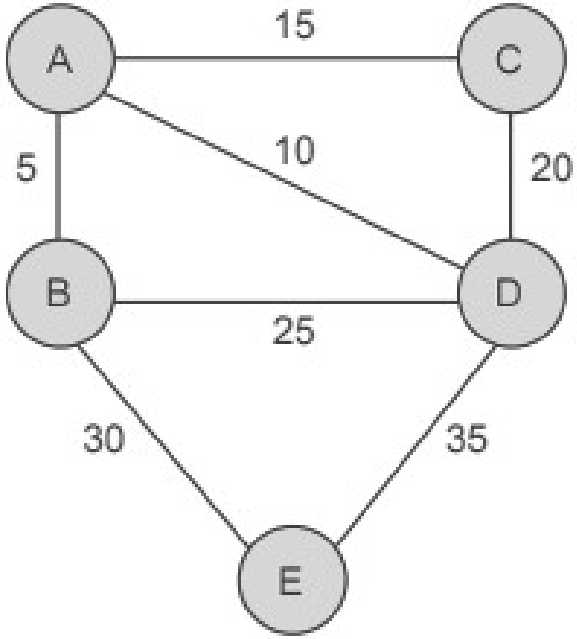
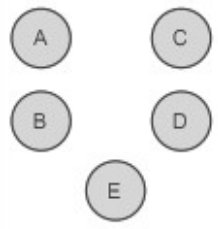
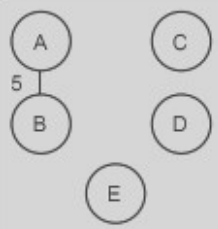
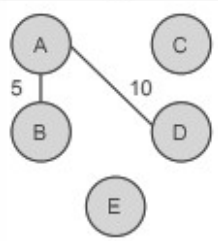
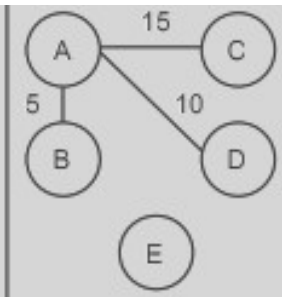
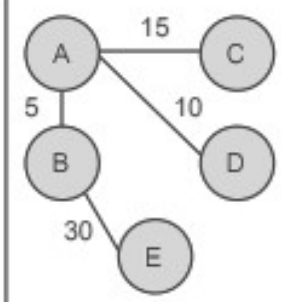


Fig. 11.16 Example graph

MST – Solution

Edges	Cost	Status	Graph present status
–	–	–	 (a)
AB	5	Accept	 (b)
AD	10	Accept	 (c)
AC	15	Accept	 (d)
CD	20	Reject	Rejected as it creates a cycle.
BD	25	Reject	Rejected as it creates a cycle.
BE	30	Accept	 (e)
DE	35	Reject	

Kruskal Algorithm

Algorithm Kruskal(G)

```
%% Input: Graph G
%% Output: MST T
Begin
      Sort the edges and form an edge list E = {e1,e2,…,en} such that
      w(e1) ≤ w(e2) ≤ … ≤ w(en)
      for all v ∈ V of G do
           T = make_set(v)                          %% Create a singleton T
      End for
           T = ϕ                                    %% Put all nodes and no edges in T
      while (T contains less than n - 1 edges) do
      Choose an edge (u, v)
      Delete the edge (u, v) from the edge list E   %% Check whether the edge
                                                    %% causes a cycle
      if (find(u)! = find(v)) then                  %% If not in the same tree then
                                                    %% add edge
           T = T ∪ T(u, v)
           DST_union(u, v)                          %% Merge trees u and v
      else
           Discard the edge
      End if
      End While
      if T has n - 1 edges, then
           return(T)
      else
           output 'No Spanning Tree is Possible.'
      End if
End
```

Kruskal Algorithm vs. Prim's Algorithm

Kruskal's algorithm	Prim's algorithm
It starts with all the graph vertices as a forest, and every addition of edge takes a forest one step towards a complete tree.	Starts with any vertex of the graph arbitrarily. At no point is a forest encountered in Prim's algorithm.
The concept of Kruskal's algorithm is based on the acyclic nature of the graph.	The concept of Prim's algorithm is based on connectedness.
More time is saved as the edges are sorted first.	It saves a lot of space instead, as no additional structures are used for storing the edges.
Adding an edge is performed by selecting the sorted edge.	A new vertex is always joined to an old vertex.
The edge list always determines the next edge.	The addition of nodes is based on the shortest distance or weight concept.

Prims Algorithm

Algorithm Prim(G)

```
%% Input: Graph G = (V, E)
%% Output: Minimum spanning tree T, T = (V_T, E_T)
Begin
      s = pick any vertex of G
      V_T = {s}
      E_T = φ                          %% Initially T(V_T, E_T) has only starting
                                       %% vertex and no edges
      n = |v|
      repeat n - 1 times               % n is the number of vertices
      pick an edge (v, u) such that  v∈V_T and u∈V − V_T
           V_T = V ∪ {v}
           E_T = E_T ∪ {v, u}
      End Repeat
      return T(V_T, E_T)
End
```

Time Complexity: $O(|E| \log|V|)$

Example: Consider the following graph (refer to Fig. 11.22) and apply the Prim's algorithm to get the minimum spanning tree

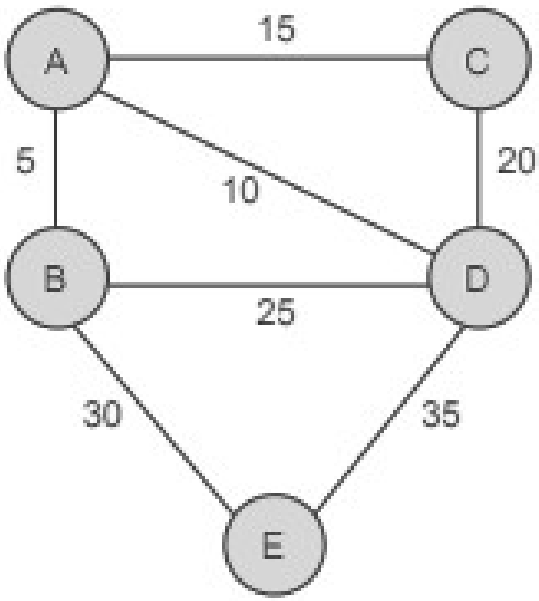
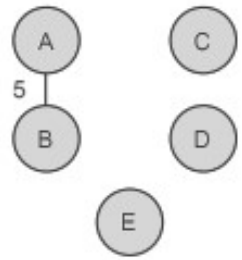
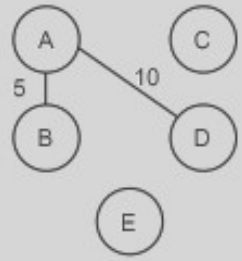
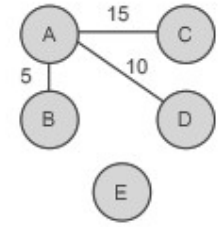
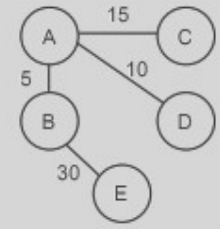


Fig. 11.20 Example graph

Solution: Prims algorithm

Edges	Minimum cost edge	Graph present status
Init the node A. So, the edges are (A, B), (A, C), and (A, D)	(A, B)	 (a)
(A, C) (A, D) (B, D) (B, E)	(A, D)	 (b)
(C, D), (D, E), (B, D), and (B, E).	(A, C)	 (c)
(C, D),(D, E),(B, D), and (B, E)	(C, D)	Rejected
(D, E),(B, D), and (B, E)	(B, D)	Rejected
(D, E) and (B, E)	(B, E)	 (d)
(D, E)	–	Rejected

Single Source Shortest Path (Djikstra's Algorithm)

Algorithm SSSP(G)

```
%% Input: weighted graph G = <V, E>
%% Output: Distance between source node s and a given node
Begin
      S = {s}
      d[s] = 0                                          %% Set the distance of s as 0
      while (S ≠ V) do
      pick a node v from V - {S} with the smallest d[v]      %% Selection procedure
           S = S U {v}
      for all the nodes v ∈ V - {S} such that (u, v) ∈ E do
           d[v] = minimum{d[v], d[u,v]}//relax
      End for
      End while
      Return d, the array that has distance from the source node s
End
```

Consider the following graph and construct SSSP from A.

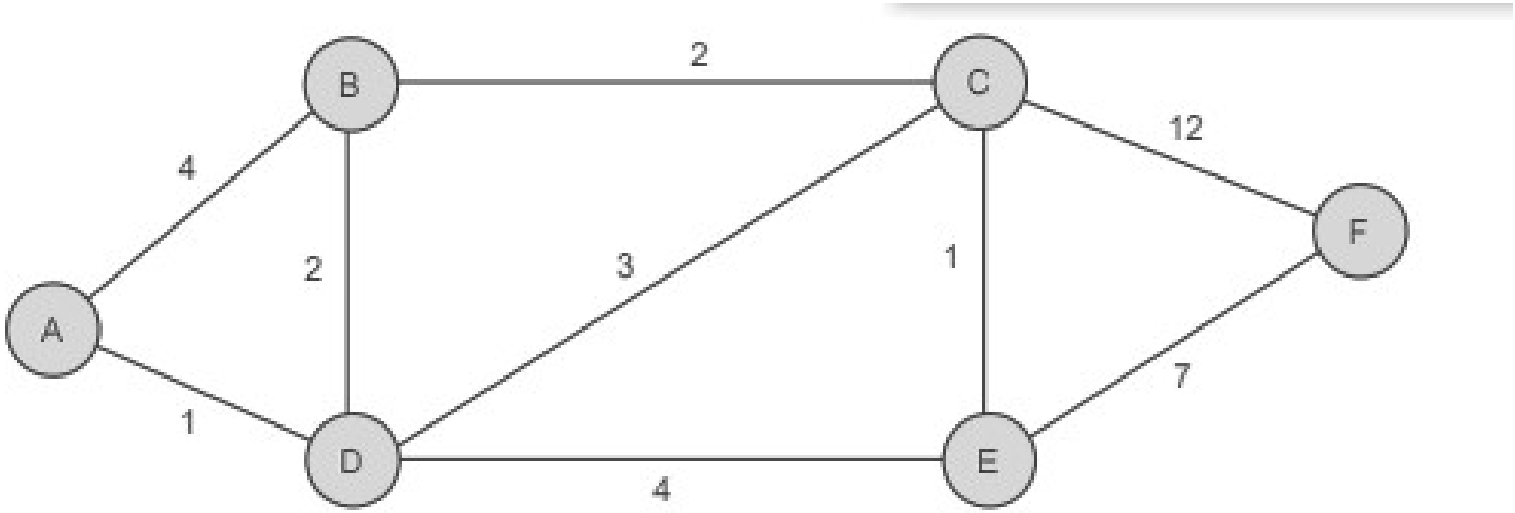


Fig. 11.24 Sample graph

SSSP Example

Initialization

A	B	C	D	E	F
0	4	1	∞	∞	∞

2. Relaxation of A

A	B	C	D	E	F
$\boxed{0}$	4	1	∞	∞	∞

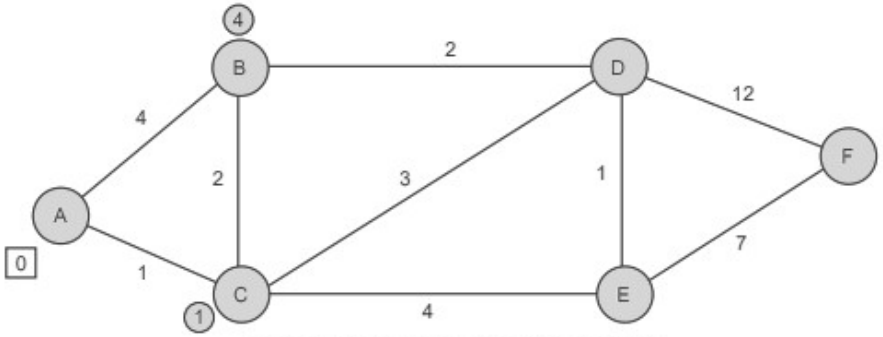


Fig. 11.24a Relaxation of all edges leaving A

3. Relaxation of C

A	B	C	D	E	F
0	3	1	4	5	∞

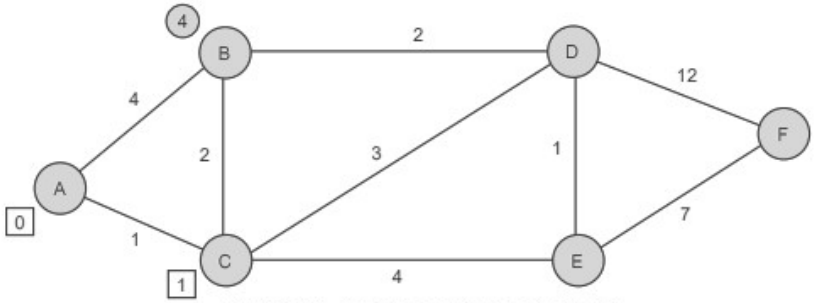


Fig. 11.24b Relaxation of all edges leaving C

4. Relaxation of B

A	B	C	D	E	F
0	3	1	4	5	∞

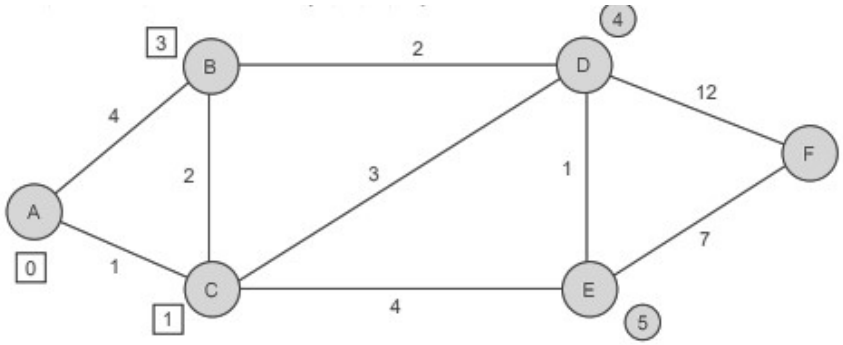


Fig. 11.24c Relaxation of all edges leaving B

5. Relaxation of D

A	B	C	D	E	F
0	3	1	4	5	16

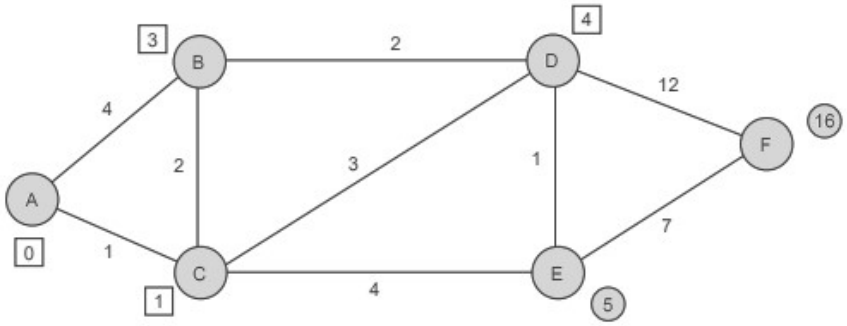


Fig. 11.24d Relaxation of all edges leaving D

6. Relaxation of E

A	B	C	D	E	F
0	3	1	4	5	12

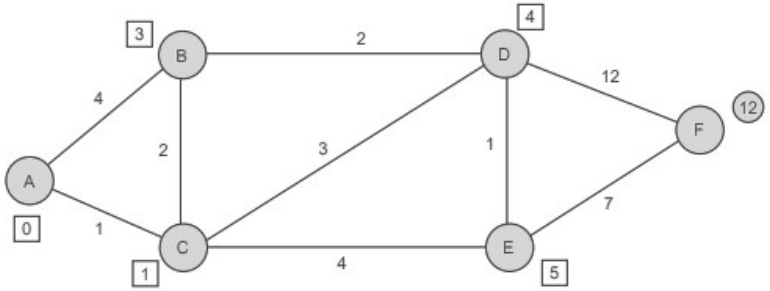


Fig. 11.24e Relaxation of all edges leaving E

7. Relaxation of F

A	B	C	D	E	F
0	3	1	4	5	12

S	d(v)						Chosen vertex
	v = A	v = B	v = C	v = D	v = E	v = F	
Init {}	0	∞	∞	∞	∞	∞	A min.
{A}	0	4	1	∞	∞	∞	C
{A, C}	0	3	1	4	5	∞	B
{A, C, B}	0	3	1	4	5	∞	D
{A, C, B, D}	0	3	1	4	5	16	E
{A, C, B, D, E}	0	3	1	4	5	12	F
{A, C, B, D, E, F}	–	–	–	–	–	–	–